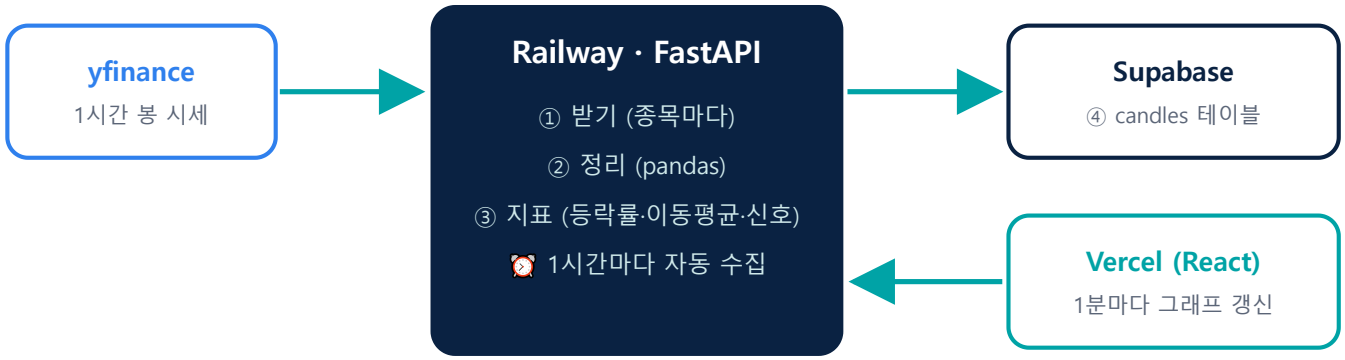
BACKEND PRACTICE · PART 2 · NO LOCAL SETUP

주식 시세를 1시간마다 모아 FastAPI로 지표를 계산하고 Supabase에 쌓아 그래프로

미세먼지 실습과 **같은 틀**에 데이터만 바꿔 끼우는 응용편
내 PC 설치 없이 **GitHub 웹 업로드 → Railway 배포** · 프론트는 1분마다 그래프 갱신



교육용 예제입니다. 백엔드 데이터 흐름을 배우기 위한 코드이며 투자 판단에 쓰지 않습니다. yfinance 시세는 15~20분 늦을 수 있고, 골든크로스 같은 신호는 계산 연습일 뿐 매수·매도 추천이 아닙니다.

제주대 바이브코딩 초중급 · 백엔드 실습 2편 (1편: 공공데이터 미세먼지)
준비물: GitHub 계정 · Railway 계정 · Supabase 프로젝트 · 이미 배포된 Vercel 프론트 (API 키 필요 없음)
코드 폴더: `code\` (main.py · collect.py · db.py · requirements.txt · .python-version · railway.json · schema.sql · StockDemo.jsx)

목차

1. 미세먼지 편과 같은 틀 · "실시간"의 뜻
2. 데이터 흐름 · 파일 · API
3. Supabase 테이블 만들기
4. GitHub 업로드와 Railway 배포
5. /docs에서 수집·조회 눌러 보기
6. Vercel에 그래프 붙이기
7. 수업용 코드 읽기 포인트
8. 오류 해결표 · 체크리스트 · 응용
9. 부록 — 전체 코드

> **1편(미세먼지)을 해 봤다면 30분이면 끝납니다.** GitHub 업로드, Railway 연결, Variables, `/docs` 테스트 순서가 똑같습니다. 다른 점은 **받는 곳(yfinance)**, **가공 내용(봉·이동평균·신호)**, **화면(그래프)** 세 가지뿐입니다.

> **이번엔 인증키가 필요 없습니다.** yfinance는 키 없이 시세를 줍니다. 그래서 Variables는 Supabase 두 개, 관리자 암호, Vercel 주소만 넣으면 됩니다.

1 미세먼지 편과 같은 틀 · “실시간”의 뜻

단계	1편 미세먼지	2편 주식 (이번)
① 받기	에어코리아 공공 API (인증키)	yfinance — 종목마다 최근 한 달치 1시간 봉 (키 없음)
② 가공	`"-"`, `24:00`, 통신장애, `-999` 정리 + 등급	빈 종가 버리기, 원화 정수로, 한국 시간 통일
③ 계산	1시간 뒤 예측 (scikit-learn)	등락률 · 5시간/20시간 이동평균 · 골든/데드크로스 · 급변동
④ 저장	`air_readings`	`candles` (같은 봉은 덮어쓰기)
⏰ 자동	60분마다	60분마다 (장 마감 후에는 같은 값을 덮어쓸 뿐)
화면	측정소 표 + 등급 색	종목 버튼 + 꺾은선 그래프, **1분마다 자동 새로고침**

왜 여기서도 FastAPI + Railway인가

- **장중 내내 혼자 모아야 합니다.** 아무도 사이트를 안 열어도 1시간마다 새 봉이 쌓여야 그래프에 구멍이 없습니다.
- **파이썬 도구가 필요합니다.** yfinance와 pandas(이동평균 `rolling`, 등락률 `pct_change`)는 브라우저에서 못 씁니다.
- **계산은 한 번만.** 서버가 지표까지 계산해 저장하면, 화면 100개가 열려도 각자 계산하지 않고 결과만 받습니다.
- **나중에 증권사 API로 바꿀 때** 그 키도 Railway Variables에만 둡니다. 화면 코드는 바뀌지 않습니다.

“실시간”은 어디까지인가

방식	얼마나 자주	이 실습과의 관계
증권사 실시간(웹소켓)	체결될 때마다 (1초에도 여러 번)	계좌·키 발급이 필요해서 이번엔 제외 (8장 응용)
yfinance 1시간 봉	1시간 단위, 15~20분 지연	**이번 실습**. 장 시간(09:00~15:30)에 하루 6~7개 봉
공공데이터포털 주식시세	하루 1번 (전날 종가)	1시간 그래프는 못 그림

미세먼지도 한 시간에 한 번 갱신되는 데이터였습니다. 공공·무료 데이터는 대부분 이렇게 **정해진 간격**으로 갱신되고, 그래서 서버가 **같은 간격으로 계속 받아 쌓는 것**이 백엔드의 일입니다. 화면의 1분 새로고침은 “서버가 새 봉을 쌓았는지 확인하는 것”입니다.

> **공개 사이트 주의:** 거래소 시세는 다시 배포하는 데 제한이 있습니다. 수업에서는 Vercel 주소를 수강생끼리만 공유하거나 연습 모드(`DEMO_MODE=1`)로 운영하세요.

2 데이터 흐름 · 파일 · API

단계	파일	하는 일
① 받기	`collect.py`	`fetch_yf()` — `yf.Ticker(종목).history(period="1mo", interval="1h")`. `DEMO_MODE=1`이면 `fetch_demo()`가 가짜 시세(평일 09~15시 봉, 빈칸·진행 중인 봉 포함)를 만듦
② 정리	`collect.py`	`clean()` 1~4단계: 필요한 칸만, 빈 종가 버리기, 원화 정수, 한국 시간 정렬
③ 지표	`collect.py`	`clean()` 5~6단계: 등락률, 이동평균(5·20), 골든/데드크로스, 급변동(`SPIKE_PCT`% 이상)
④ 저장	`db.py`	`candles`에 덮어쓰기 저장(500개씩 나눠서), 수집 기록은 `stock_logs`
⏰·🔔	`main.py`	60분마다 자동 수집 + API 주소들. 한 종목이 실패해도 나머지는 계속

```
code/
├─ main.py            # API 주소들 + 1시간마다 자동 수집
├─ collect.py         # ① 받기 + ② 정리 + ③ 지표
├─ db.py              # ④ Supabase 저장 (변수 없으면 메모리 임시 저장)
├─ requirements.txt   # fastapi · uvicorn · pandas · yfinance · supabase
├─ .python-version    # 3.12
├─ railway.json       # 시작 명령 + 헬스체크
├─ schema.sql         # Supabase에 붙여 넣을 테이블
└─ StockDemo.jsx      # Vercel 프론트에 넣을 그래프 화면
```

API 한눈에

주소	방식	돌려주는 것
`/health`	GET	모드(yfinance/demo), 저장소, 종목 목록, 빠진 환경변수
`/collect`	POST	지금 바로 1회 수집. 헤더 `x-admin-token`. 받은 봉·버린 봉·저장 수·실패 종목
`/stocks`	GET	종목별 마지막 봉 (가격·등락률) — 버튼 목록용
`/stocks/005930.KS/candles?limit=120`	GET	그래프용 1시간 봉 (오래된 → 최신)
`/stocks/005930.KS/signals`	GET	골든크로스·데드크로스·급변동이 있었던 봉만 (최근 30개)
`/logs`	GET	최근 수집 기록 20개

종목 코드는 야후 형식입니다: 코스피 `005930.KS`(삼성전자), 코스닥은 `.KQ`. 기본 종목: 삼성전자·SK하이닉스·NAVER.

3 Supabase 테이블 만들기

Supabase → **SQL Editor** → **New query** → 붙여 넣고 **Run**. 1편과 같은 프로젝트를 써도 됩니다. 테이블 이름이 달라서 섞이지 않습니다.

> 수집 기록 테이블 이름은 `stock_logs` 라서 1편의 `collect_logs` 와 겹치지 않습니다.

schema.sql

```
-- Supabase → SQL Editor 에 붙여 넣고 Run
create table if not exists candles (
  id          bigint generated always as identity primary key,
  source      text not null,                -- 'yfinance'(진짜) / 'demo'(연습)
  ticker      text not null,                -- 005930.KS
  name        text,                         -- 삼성전자
  ts          timestamptz not null,         -- 봉 시작 시각 (1시간 단위)
  open        bigint, high bigint, low bigint, close bigint,
  volume      bigint,
  change_pct  real,                         -- 직전 봉 대비 등락률 %  (FastAPI가 계산)
  ma5         real,                         -- 5시간 이동평균
  ma20        real,                         -- 20시간 이동평균
  signal      text,                         -- 골든크로스 / 데드크로스 / 급변동
  collected_at timestamptz not null default now(),
  unique (source, ticker, ts)               -- 같은 봉은 덮어쓰기 (중복 방지)
);

create table if not exists stock_logs (
  id       bigint generated always as identity primary key,
  ran_at   timestamptz not null default now(),
  source   text,
  tickers  int,      -- 종목 수
  fetched  int,      -- 받은 봉 수
  dropped  int,      -- 버린 봉 수
  cleaned  int,      -- 정리된 봉 수
  saved    int,      -- 저장(덮어쓰기 포함)된 봉 수
  error    text      -- 실패한 종목 메시지
);

-- 브라우저에서 직접 못 건드리게 잠금. FastAPI는 secret 키라서 통과.
alter table candles enable row level security;
alter table stock_logs enable row level security;
```

Railway에 넣을 값: **Project Settings → Data API**의 Project URL(`SUPABASE_URL`), **API Keys**의 Secret key `sb_secret_…` (`SUPABASE_SECRET_KEY`). secret 키는 서버 전용입니다.

4 GitHub 업로드와 Railway 배포

4-1. GitHub (브라우저만)

1. github.com → **New repository** → 이름 `stock-api` · Private → Create
2. **uploading an existing file** → `code` 폴더 **안의 파일들**을 끌어다 놓기 → **Commit changes**
3. `.python-version`이 안 보이면 탐색기 **보기 → 숨긴 항목** 체크, 또는 GitHub에서 **Add file → Create new file** 로 직접 만들기(내용 `3.12`)

4-2. Railway

1. railway.com → **New Project → Deploy from GitHub repo** → `stock-api`
2. 서비스 → **Variables → Raw Editor**에 붙여 넣고 값 바꾸기 → **Update Variables** → 위에 뜨는 **Deploy**

```
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_SECRET_KEY=sb_secret_xxxxxxxxxxxxxxxx
ADMIN_TOKEN=stock-아무도모르는-긴문자열-2026
FRONT_ORIGINS=https://my-app.vercel.app
TICKERS=005930.KS,000660.KS,035420.KS
DEMO_MODE=0
COLLECT_MINUTES=60
SPIKE_PCT=1.5
```

변수	뜻
`TICKERS`	모을 종목(쉼표). 종목이 많을수록 수집 시간이 길어지니 수업에선 3~5개
`DEMO_MODE`	`1`이면 가짜 시세. yfinance가 막히거나 주말에 움직임을 보여 주고 싶을 때
`SPIKE_PCT`	한 시간에 몇 % 이상 움직이면 "급변동"으로 표시할지
`COLLECT_MINUTES`	자동 수집 간격(분). 1시간 봉이므로 60이면 충분, 더 빨리 반영하려면 15~30

3. **Settings → Networking → Generate Domain** → `https://…/health` 확인

```
{"ok":true,"mode":"yfinance","store":"supabase",
 "tickers":["005930.KS","000660.KS","035420.KS"],"auto_collect_minutes":60,"missing":[]}
```

> **Serverless(App Sleeping)는 끄세요.** 서버가 자면 1시간마다 하는 수집도 멈춥니다.

5 /docs에서 수집·조회 눌러 보기

`https://…/docs`를 엽니다. 서버가 켜지면 바로 1회 수집하므로, 대개 이미 데이터가 들어 있습니다.

5-1. 수집 기록 — `GET /logs`


```
[{"source":"yfinance","tickers":3,"fetched":396,"dropped":0,"cleaned":396,"saved":396,"error":null}]
```


실제 실행 예(3종목 × 약 132개 봉 = 한 달치). 연습 모드에서는 `"fetched":462, "dropped":3`처럼 일부러 넣은 빈 봉이 버려지는 게 보입니다.

> **수업 포인트:** 1시간마다 다시 수집해도 `saved`는 늘지만 Supabase 행 수는 거의 그대로입니다. 같은 종목·같은 시각은 **덮어쓰기(upsert)**라서, 새로 생긴 봉만 행이 늘어납니다. Table Editor에서 행 수를 비교해 보세요.

5-2. 지금 바로 수집 — `POST /collect`

Try it out → **x-admin-token**에 `ADMIN_TOKEN` 값 → Execute. 응답의 `"error"`에 종목별 실패 이유가 들어 있습니다(없으면 `null`).

5-3. 그래프 데이터 — `GET /stocks/{ticker}/candles`

ticker에 `005930.KS`, **limit**에 `3` → Execute


```
{"ticker":"005930.KS","name":"삼성전자","mode":"yfinance","candles":[
 {"time":"2026-11-10 13:00","close":198300,"change_pct":0.15,"ma5":198670.0,"ma20":194778.0,"signal":
null},
 {"time":"2026-11-10 14:00","close":199600,"change_pct":0.66,"ma5":198830.0,"ma20":194558.0,"signal":
null}, …]}
```


값은 실행하는 날 시세에 따라 다릅니다. 응답의 `open·high·low·volume`은 줄였습니다.

5-4. 신호만 — `GET /stocks/{ticker}/signals`

골든크로스(5시간 평균이 20시간 평균을 위로 뚫음), 데드크로스(아래로 뚫음), 급변동(한 시간에 `SPIKE_PCT` % 이상)이 표시된 봉만 나옵니다. 그래프의 색 점과 같습니다.

6 Vercel에 그래프 붙이기

1. Vercel → Settings → Environment Variables: `VITE_API_URL` = `https://stock-api-production-xxxx.up.railway.app` (끝 / 없이) → **Redeploy**
2. `StockDemo.jsx`를 프론트 `src`에 넣고 `App.jsx`에서 `import StockDemo from "./StockDemo";` → `<StockDemo />`
3. Railway `FRONT_ORIGINS`에 Vercel 주소가 있는지 확인 (CORS)

그래프 라이브러리를 설치하지 않도록 **SVG로 직접** 그렸습니다. 종가(실선), 5시간·20시간 평균(점선), 신호(색 점)를 그리고 **1분마다** 다시 물어봅니다.

src/StockDemo.jsx

```
// Vercel 프론트(React)에 넣는 화면 — 그래프용 데이터만 Railway에 물어본다 (라이브러리 없이 SVG)
import { useEffect, useState } from "react";

const API = import.meta.env.VITE_API_URL;
const W = 640, H = 300, PAD = 40;

function Chart({ candles }) {
  if (!candles.length) return null;
  const vals = candles.flatMap((c) => [c.close, c.ma5, c.ma20]).filter((v) => v != null);
  const min = Math.min(...vals), max = Math.max(...vals);
  const x = (i) => PAD + (i * (W - PAD * 2)) / Math.max(1, candles.length - 1);
  const y = (v) => H - PAD - ((v - min) / Math.max(1, max - min)) * (H - PAD * 2);
  const line = (key) =>
    candles.map((c, i) => (c[key] == null ? null : `${x(i)},${y(c[key])}`)).filter(Boolean).join("
");
  const mark = { 골든크로스: "#27ae60", 데드크로스: "#eb5757", 급변동: "#f2994a" };

  return (
    <svg viewBox={`0 0 ${W} ${H}`} style={{ width: "100%", background: "#fff", border: "1px solid #dd
d" }}>
      <text x={PAD} y={20} fontSize="12">{max.toLocaleString()}원</text>
      <text x={PAD} y={H - 8} fontSize="12">{min.toLocaleString()}원</text>
      <polyline points={line("close")} fill="none" stroke="#0d8784" strokeWidth="2" />
      <polyline points={line("ma5")} fill="none" stroke="#3a82d2" strokeWidth="1.5" strokeDasharray
="4 3" />
      <polyline points={line("ma20")} fill="none" stroke="#8e6bc9" strokeWidth="1.5" strokeDasharray
="4 3" />
      {candles.map((c, i) =>
        c.signal ? <circle key={i} cx={x(i)} cy={y(c.close)} r="4" fill={mark[c.signal]} /> : null
      )}
    </svg>
  );
}

export default function StockDemo() {
  const [list, setList] = useState([]);
  const [ticker, setTicker] = useState("005930.KS");
  const [data, setData] = useState(null);
  const [error, setError] = useState("");

  async function load() {
    try {
```

```
      const [l, d] = await Promise.all([
        fetch(`${API}/stocks`).then((r) => r.json()),
        fetch(`${API}/stocks/${ticker}/candles?limit=120`).then((r) => r.json()),
      ]);
      setList(l);
      setData(d);
      setError("");
    } catch (e) {
      setError(String(e));
    }
  }

  // 처음 + 종목을 바꿀 때 + 1분마다 다시 물어보기 (서버는 1시간마다 새 봉을 저장)
  useEffect(() => {
    load();
    const id = setInterval(load, 60_000);
    return () => clearInterval(id);
  }, [ticker]);

  const last = data?.candles?.at(-1);
  return (
    <div style={{ padding: 20, maxWidth: 680 }}>
      <h3>주식 1시간 봉 (yfinance → FastAPI 가공 → Supabase)</h3>
      <p style={{ fontSize: 12, color: "#888" }}>교육용 예제 · 시세는 늦을 수 있음 · 투자 판단용 아님</p
>
      {error && <p style={{ color: "red" }}>{error}</p>}
      <div style={{ display: "flex", gap: 8, flexWrap: "wrap" }}>
        {list.map((s) => (
          <button key={s.ticker} onClick={() => setTicker(s.ticker)}
            style={{ fontWeight: s.ticker === ticker ? 700 : 400 }}>
            {s.name} {s.ready && <span style={{ color: s.change_pct >= 0 ? "#d8443a" : "#3a82d2" }}>
              {s.change_pct >= 0 ? "▲" : "▼"}{Math.abs(s.change_pct ?? 0)}%</span>}
          </button>
        ))}
      </div>
      {last && (
        <p>
          <b>{data.name}</b> {last.close.toLocaleString()}원 · {last.time} 봉
          {data.mode === "demo" && <small> (연습 데이터)</small>}
        </p>
      )}
      {data?.candles && <Chart candles={data.candles} />}
      <p style={{ fontSize: 12 }}>
        <span style={{ color: "#0d8784" }}>━ 종가</span>{" "}
        <span style={{ color: "#3a82d2" }}>┅ 5시간 평균</span>{" "}
        <span style={{ color: "#8e6bc9" }}>┅ 20시간 평균</span>{" "}
        ● 신호(골든·데드크로스, 급변동)
      </p>
    </div>
  );
}
```

프론트에는 계산 코드가 없습니다. 이동평균도 신호도 서버가 이미 계산해 DB에 넣어 두었고, 화면은 받은 숫자로 선을 그릴 뿐입니다. 1편과 같은 역할 분리입니다.

7 수업용 코드 읽기 포인트

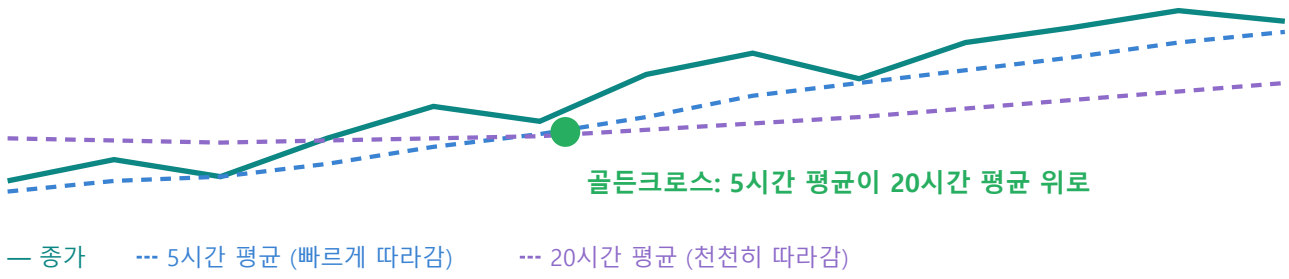
7-1. 1시간 봉이란

한 시간 동안의 가격을 네 숫자로 줄인 것입니다: **시가**(처음), **고가**(가장 높음), **저가**(가장 낮음), **종가**(마지막). 그래프는 보통 종가를 잇습니다. 장중 마지막 봉은 아직 **진행 중**이라 다음 수집 때 값이 바뀌고, 같은 시각이라 덮어써집니다.

7-2. `clean()` 일곱 단계

#	코드	왜
1	필요한 칸만, 소문자 이름	yfinance는 배당·분할 칸도 줌 → DB 칸과 맞춤
2	빈 종가·0 이하 버리기	거래 정지·데이터 누락 봉은 선을 망침
3	원화 → 정수	`199600.0` 같은 소수점 제거
4	한국 시간으로 통일 · 시간순 정렬	DB와 화면 시각을 맞춤
5	`pct_change()`, `rolling(5)`, `rolling(20)`	등락률·이동평균을 한 줄로 (pandas의 힘)
6	평균선 위아래가 바뀐 순간 = 크로스, 큰 등락 = 급변동	"계산된 정보"를 함께 저장
7	DB 행(dict)으로	빈값(NaN)은 `None`으로 바꿔야 저장됨

7-3. 이동평균과 크로스 (그림)




```
above   = ma5 > ma20                  # 지금 빠른 선이 위인가?
crossed = above != above.shift()      # 직전 봉과 위아래가 바뀌었나?
crossed & above   → "골든크로스"
crossed & ~above  → "데드크로스"
```

신호는 "이런 계산을 서버에서 한다"는 예시일 뿐, 맞는지 틀리는지 보장하지 않습니다.

7-4. 한 종목이 실패해도

```
for t in TICKERS:
    try:    받기 → 정리 → 저장
    except Exception as e:
        errors.append(...)    # 기록만 하고 다음 종목으로
```

잘못된 종목 코드 하나 때문에 전체 수집이 멈추지 않게 하는 서버 코드의 기본 습관입니다. 실패 이유는 `stock_logs.error`에 남습니다.

8 오류 해결표 · 체크리스트 · 응용

증상	원인	해결
`/logs` 의 error에 `Too Many Requests` / `Rate limited`	야후가 Railway 같은 클라우드 서버 요청을 일시적으로 막음	잠시 뒤 `/collect` 재시도, 종목 수 줄이기, 수업 중엔 `DEMO_MODE=1`
error에 `시세가 비어 있습니다`	종목 코드 오타(`.KS` / `.KQ`) 또는 상장폐지	finance.yahoo.com에서 코드 확인
빌드 실패 `No start command`	파일이 `code/` 폴더째 올라감	저장소 맨 위에 `main.py` 가 오게 다시 업로드
`no unique or exclusion constraint`	테이블에 unique 줄이 빠짐	schema.sql 그대로 다시 만들기
`/health` 의 store가 `memory(임시)`	SUPABASE 변수 누락 / Deploy 안 누름	Variables 확인 → Deploy
`/stocks/…/candles` 404	그 종목이 TICKERS에 없거나 수집 전	TICKERS 확인, `/collect` 실행
그래프가 주말 내내 안 움직임	장이 닫혀 새 봉이 없음 (정상)	움직임을 보여 주려면 `DEMO_MODE=1`
ma20이 처음 몇 개 `null`	20개가 쌓여야 평균을 낼 수 있음 (정상)	—
프론트 `Failed to fetch` + CORS	`FRONT_ORIGINS` 불일치	Vercel 주소 그대로, 끝 / 없이 → Deploy

체크리스트

- ☐ Supabase에 `candles` , `stock_logs` 테이블 (RLS 켜짐)
- ☐ GitHub 저장소 맨 위에 `main.py` , `requirements.txt` , `railway.json` , `.python-version`
- ☐ Railway Variables: SUPABASE_URL · SUPABASE_SECRET_KEY · ADMIN_TOKEN · FRONT_ORIGINS · TICKERS
- ☐ `/health` → `"store":"supabase"` , `/logs` → `"error": null`
- ☐ `/stocks/005930.KS/candles` 에 봉이 120개 안팎
- ☐ Vercel `VITE_API_URL` 추가 후 재배포, 그래프와 종목 버튼 동작

응용 과제

바꿀 곳	아이디어
① 받기	한국투자증권 Open API(모의투자 계좌)로 바꾸기 → 진짜 실시간. 키는 Variables에만, 화면 코드는 그대로
② 가공	시간 봉을 모아 **일봉** 만들기(`resample("1D")`), 거래량 급증 표시
③ 계산	RSI 같은 다른 지표, 1편처럼 scikit-learn으로 다음 봉 예측 후 **얼마나 틀리는지** 기록
화면	Supabase Realtime으로 새 봉이 저장되는 순간 그래프에 바로 반영 (1분 새로고침 대신)
합치기	1편 미세먼지 + 2편 주식을 한 Railway 서비스에 → "수집기 여러 개를 가진 백엔드"

A 부록 — 전체 코드

main.py

```
"""
main.py — 주식 1시간 봉 백엔드 (FastAPI)  ※ 교육용 예제, 투자 판단용 아님

   yfinance ──① 받기──▶ FastAPI ──② 정리·지표 계산──▶ ④ Supabase 저장
                            ▲
   Vercel(React) ── 그래프용 데이터만 요청 ──┘

Railway Variables
  SUPABASE_URL, SUPABASE_SECRET_KEY   Supabase 연결 (없으면 메모리 임시 저장)
  ADMIN_TOKEN                         /collect 를 아무나 못 누르게 하는 암호
  FRONT_ORIGINS                       Vercel 주소 (쉼표로 여러 개)
  TICKERS=005930.KS,000660.KS,035420.KS   종목 (야후 코드: 코스피 .KS · 코스닥 .KQ)
  DEMO_MODE=0     1 이면 가짜 시세 (yfinance가 막혔을 때)
  COLLECT_MINUTES=60  SPIKE_PCT=1.5  AUTO_COLLECT=1
"""
import asyncio
import logging
import os
from contextlib import asynccontextmanager
from datetime import datetime, timezone

from fastapi import FastAPI, Header, HTTPException
from fastapi.middleware.cors import CORSMiddleware

import collect
import db

log = logging.getLogger("uvicorn.error")

TICKERS = [t.strip().upper() for t in os.getenv("TICKERS", "005930.KS,000660.KS,035420.KS").split
(",") if t.strip()]
DEMO = os.getenv("DEMO_MODE", "0") == "1"
SOURCE = "demo" if DEMO else "yfinance"
ADMIN_TOKEN = os.getenv("ADMIN_TOKEN", "").strip()
AUTO = os.getenv("AUTO_COLLECT", "1") == "1"
MINUTES = int(os.getenv("COLLECT_MINUTES", "60"))
SPIKE = float(os.getenv("SPIKE_PCT", "1.5"))
FRONT_ORIGINS = [o.strip().rstrip("/") for o in os.getenv("FRONT_ORIGINS", "").split(",") if o.strip
()]

store = db.get_store()


# ──────────── 수집 한 번 = 종목마다 받기 → 정리·지표 → 저장, 그리고 기록 ────────────
def run_collect() -> dict:
    total = {"fetched": 0, "dropped": 0, "cleaned": 0, "saved": 0}
    errors = []
    for t in TICKERS:
        try:
            raw = collect.fetch_demo(t) if DEMO else collect.fetch_yf(t)
            rows, st = collect.clean(raw, t, SOURCE, SPIKE)
            saved = store.save_candles(rows)
```

```
            for k in ["fetched", "dropped", "cleaned"]:
                total[k] += st[k]
            total["saved"] += saved
        except Exception as e:                      # 한 종목이 실패해도 나머지는 계속
            errors.append(f"{t}: {e}"[:200])
    entry = {"ran_at": datetime.now(timezone.utc).isoformat(), "source": SOURCE,
             "tickers": len(TICKERS), **total, "error": "; ".join(errors) or None}
    store.save_log(entry)
    log.info("collect %s", entry)
    return entry


async def auto_loop():
    """서버가 켜져 있는 동안 COLLECT_MINUTES 마다 자동 수집 (브라우저가 꺼져 있어도)"""
    while True:
        try:
            await asyncio.to_thread(run_collect)
        except Exception:
            log.exception("자동 수집 실패")
        await asyncio.sleep(MINUTES * 60)


@asynccontextmanager
async def lifespan(app: FastAPI):
    task = asyncio.create_task(auto_loop()) if AUTO else None
    yield
    if task:
        task.cancel()


app = FastAPI(title="주식 1시간 봉 백엔드 (교육용)", lifespan=lifespan)
app.add_middleware(CORSMiddleware, allow_origins=FRONT_ORIGINS,
                   allow_methods=["*"], allow_headers=["*"])


def _view(r):
    return {"time": db.kst(r["ts"]), "open": r["open"], "high": r["high"], "low": r["low"],
            "close": r["close"], "volume": r["volume"], "change_pct": r["change_pct"],
            "ma5": r["ma5"], "ma20": r["ma20"], "signal": r["signal"]}


@app.get("/")
def home():
    return {"service": "주식 1시간 봉 백엔드 (교육용)", "mode": SOURCE, "docs": "/docs"}


@app.get("/health")
def health():
    missing = [k for k in ["SUPABASE_URL", "SUPABASE_SECRET_KEY", "ADMIN_TOKEN", "FRONT_ORIGINS"]
               if not os.getenv(k, "").strip()]
    return {"ok": True, "mode": SOURCE, "store": store.kind, "tickers": TICKERS,
            "auto_collect_minutes": MINUTES if AUTO else None, "missing": missing}


@app.post("/collect")
def collect_now(x_admin_token: str = Header(default="")):
    """지금 바로 한 번 수집 (Swagger /docs 에서 Try it out)"""
```

```
    if not ADMIN_TOKEN:
        raise HTTPException(503, "Railway Variables에 ADMIN_TOKEN을 먼저 넣으세요")
    if x_admin_token != ADMIN_TOKEN:
        raise HTTPException(401, "x-admin-token이 틀렸습니다")
    return run_collect()


@app.get("/stocks")
def stocks():
    """종목별 마지막 봉 요약 (목록 화면용)"""
    out = []
    for t in TICKERS:
        rs = store.candles(SOURCE, t, 2)
        if not rs:
            out.append({"ticker": t, "name": collect.NAMES.get(t, t), "ready": False})
            continue
        last = rs[-1]
        out.append({"ticker": t, "name": last["name"], "ready": True, **_view(last)})
    return out


@app.get("/stocks/{ticker}/candles")
def candles(ticker: str, limit: int = 120):
    """그래프용 1시간 봉 (오래된 것 → 최신 순)"""
    rs = store.candles(SOURCE, ticker.upper(), min(max(limit, 1), 500))
    if not rs:
        raise HTTPException(404, f"{ticker} 데이터가 없습니다 (TICKERS 확인 · /collect 실행)")
    return {"ticker": ticker.upper(), "name": rs[-1]["name"], "mode": SOURCE,
            "candles": [_view(r) for r in rs]}


@app.get("/stocks/{ticker}/signals")
def signals(ticker: str):
    """골든크로스 · 데드크로스 · 급변동이 있었던 시각만"""
    rs = store.candles(SOURCE, ticker.upper(), 500)
    return [_view(r) for r in rs if r["signal"]][-30:]


@app.get("/logs")
def logs():
    return store.logs()
```

collect.py

```
"""
collect.py — ① 시세 받기  ② 가공(정리 + 지표 계산)

yfinance로 종목별 '1시간 봉'(시가·고가·저가·종가·거래량)을 받아서
pandas로 정리하고 등락률 · 이동평균 · 신호를 계산해 DB에 넣을 행으로 만든다.
※ 교육용 예제입니다. yfinance 시세는 15~20분 늦을 수 있고, 투자 판단용이 아닙니다.
"""
import random
from datetime import datetime, timedelta, timezone

import pandas as pd

KST = timezone(timedelta(hours=9))
```

```
NAMES = {"005930.KS": "삼성전자", "000660.KS": "SK하이닉스", "035420.KS": "NAVER",
         "035720.KS": "카카오", "005380.KS": "현대차"}


# ──────────────────────────────────────────────
# ① 받기 — yfinance (키 필요 없음)
# ──────────────────────────────────────────────
def fetch_yf(ticker: str) -> pd.DataFrame:
    """최근 한 달치 1시간 봉. 매번 한 달치를 받아 덮어쓰면 이동평균이 항상 정확하다."""
    import yfinance as yf
    df = yf.Ticker(ticker).history(period="1mo", interval="1h")
    if df.empty:
        raise RuntimeError(f"{ticker}: 시세가 비어 있습니다 (종목코드·장 휴일 확인)")
    return df


# ──────────────────────────────────────────────
# ①' 연습용 가짜 시세 — yfinance가 막혀도 실습할 수 있게
# ──────────────────────────────────────────────
def fetch_demo(ticker: str) -> pd.DataFrame:
    rnd = random.Random(ticker)                       # 종목마다 늘 같은 흐름
    price = {"005930.KS": 70000, "000660.KS": 180000}.get(ticker, 50000) * 1.0
    now = datetime.now(KST)
    day = (now - timedelta(days=31)).replace(hour=0, minute=0, second=0, microsecond=0)
    rows = []
    while day <= now:
        if day.weekday() < 5:                         # 평일만 장이 열림
            for h in range(9, 16):                    # 09시 ~ 15시 봉
                ts = day.replace(hour=h)
                if ts > now:
                    break
                o = price
                c = o * (1 + rnd.gauss(0.0003, 0.008))
                hi = max(o, c) * (1 + abs(rnd.gauss(0, 0.003)))
                lo = min(o, c) * (1 - abs(rnd.gauss(0, 0.003)))
                vol = int(abs(rnd.gauss(1, 0.4)) * 500000)
                rows.append({"Datetime": ts, "Open": o, "High": hi, "Low": lo, "Close": c, "Volume":
vol})
                price = c
        day += timedelta(days=1)
    df = pd.DataFrame(rows).set_index("Datetime")
    # 진짜 데이터처럼 '덜 끝난 봉(거래량 0)'과 빈칸을 일부러 섞는다
    if len(df) > 10:
        df.iloc[-1, df.columns.get_loc("Volume")] = 0
        df.iloc[len(df) // 2, df.columns.get_loc("Close")] = float("nan")
    return df


# ──────────────────────────────────────────────
# ② 가공 — 정리 + 지표
# ──────────────────────────────────────────────
def clean(df: pd.DataFrame, ticker: str, source: str, spike_pct: float = 1.5):
    stats = {"fetched": len(df), "dropped": 0, "cleaned": 0}
    df = df.copy()

    # 1) 필요한 칸만, 이름은 소문자로
```

```
    df = df[["Open", "High", "Low", "Close", "Volume"]].rename(columns=str.lower)

    # 2) 종가가 비었거나 0 이하인 봉은 버린다
    before = len(df)
    df = df[df["close"].notna() & (df["close"] > 0)]
    stats["dropped"] = before - len(df)

    # 3) 원화는 소수점이 없다 → 정수로
    for col in ["open", "high", "low", "close"]:
        df[col] = df[col].round().astype("int64")
    df["volume"] = df["volume"].fillna(0).astype("int64")

    # 4) 시각을 한국 시간으로 통일하고 순서대로
    idx = pd.to_datetime(df.index)
    idx = idx.tz_localize(KST) if idx.tz is None else idx.tz_convert(KST)
    df.index = idx
    df = df.sort_index()

    # 5) 지표: 직전 봉 대비 등락률, 5시간·20시간 이동평균
    df["change_pct"] = (df["close"].pct_change() * 100).round(2)
    df["ma5"] = df["close"].rolling(5).mean().round(0)
    df["ma20"] = df["close"].rolling(20).mean().round(0)

    # 6) 신호: 이동평균이 엇갈린 순간 + 한 시간에 크게 움직인 순간
    above = df["ma5"] > df["ma20"]
    crossed = above.ne(above.shift()) & df["ma20"].notna() & df["ma20"].shift().notna()
    df["signal"] = None
    df.loc[crossed & above, "signal"] = "골든크로스"
    df.loc[crossed & ~above, "signal"] = "데드크로스"
    df.loc[df["signal"].isna() & (df["change_pct"].abs() >= spike_pct), "signal"] = "급변동"

    # 7) DB 행으로. 아직 진행 중인 마지막 봉도 넣지만, 다음 수집 때 같은 시각으로 덮어써진다
    rows = []
    for ts, r in df.iterrows():
        rows.append({
            "source": source, "ticker": ticker, "name": NAMES.get(ticker, ticker),
            "ts": ts.isoformat(),
            "open": int(r.open), "high": int(r.high), "low": int(r.low), "close": int(r.close),
            "volume": int(r.volume),
            "change_pct": None if pd.isna(r.change_pct) else float(r.change_pct),
            "ma5": None if pd.isna(r.ma5) else float(r.ma5),
            "ma20": None if pd.isna(r.ma20) else float(r.ma20),
            "signal": r.signal if isinstance(r.signal, str) else None,
        })
    stats["cleaned"] = len(rows)
    return rows, stats
```

db.py

```
"""
db.py — ④ 저장: Supabase (변수가 없으면 서버 메모리에 임시 저장)
"""
import os
from datetime import datetime, timedelta, timezone

KST = timezone(timedelta(hours=9))
```

```
def kst(ts: str) -> str:
    return datetime.fromisoformat(ts).astimezone(KST).strftime("%Y-%m-%d %H:%M")


class SupabaseStore:
    kind = "supabase"

    def __init__(self, url, key):
        from supabase import create_client
        self.sb = create_client(url, key)

    def save_candles(self, rows):
        if not rows:
            return 0
        saved = 0
        for i in range(0, len(rows), 500):           # 한 번에 너무 많이 보내지 않게 나눠서
            res = (self.sb.table("candles")
                   .upsert(rows[i:i + 500], on_conflict="source,ticker,ts").execute())
            saved += len(res.data or [])
        return saved

    def save_log(self, log):
        self.sb.table("stock_logs").insert(log).execute()

    def candles(self, source, ticker, limit=150):
        res = (self.sb.table("candles").select("*").eq("source", source).eq("ticker", ticker)
               .order("ts", desc=True).limit(limit).execute())
        return list(reversed(res.data))

    def logs(self, limit=20):
        res = (self.sb.table("stock_logs").select("*")
               .order("ran_at", desc=True).limit(limit).execute())
        return res.data


class MemoryStore:
    kind = "memory(임시)"

    def __init__(self):
        self.rows, self._logs = {}, []

    def save_candles(self, rows):
        for r in rows:
            self.rows[(r["source"], r["ticker"], r["ts"])] = r
        return len(rows)

    def save_log(self, log):
        self._logs.append(log)

    def candles(self, source, ticker, limit=150):
        rs = [r for r in self.rows.values() if r["source"] == source and r["ticker"] == ticker]
        return sorted(rs, key=lambda r: r["ts"])[-limit:]

    def logs(self, limit=20):
        return list(reversed(self._logs))[:limit]
```

```
def get_store():
    url = os.getenv("SUPABASE_URL", "").strip()
    key = os.getenv("SUPABASE_SECRET_KEY", "").strip()
    return SupabaseStore(url, key) if url and key else MemoryStore()
```

requirements.txt

```
fastapi
uvicorn[standard]
pandas
yfinance
supabase
```

.python-version

```
3.12
```

railway.json

```
{
  "$schema": "https://railway.com/railway.schema.json",
  "deploy": {
    "startCommand": "uvicorn main:app --host 0.0.0.0 --port $PORT",
    "healthcheckPath": "/health",
    "restartPolicyType": "ON_FAILURE"
  }
}
```